POSTGRADUATE DIPLOMA IN ICT

Capstone Project: Implementation & Testing/Evaluation Submission

Project Title	Agentic AI-Based Development Tool for South African Software Industry (LegacyGuard)
Student Full Name	Sibabalwe Ngandana
Student Number	220193894
Supervisor Name	Dr Didi
Stream Category	Application Development
Academic Year	2026
Submission Date	7 November 2026
Repository / Code Location	https://github.com/Ngandana/LegacyGuard.git

Declaration

I declare that the implementation, testing, and evaluation described in this submission is my own original work and has been prepared in accordance with the academic integrity policies of the institution.

Student Signature	S.Ngandana
Date	7 November 2026

1. Purpose of This Document

This document accompanies your implemented system and its source code/repository. It is assessed against the Implementation & Testing/Evaluation Rubric. Where a section refers to the codebase, ensure the referenced code is present and accessible in the repository/archive submitted alongside this document.

This document accompanies the LegacyGuard implementation, whose source code is in the repository named on the cover page. The Design Phase specified a pipeline in which static analysis detects vulnerabilities in legacy Java / Spring Boot backends, a large language model (LLM) agent explains each finding and proposes a fix, the fix is staged and verified on an isolated branch, and only a human reviewer can merge it, with every action recorded in an audit trail. The pipeline targets four weakness classes: hard-coded credentials (CWE-798), weak or plaintext password storage (CWE-916/256), SQL injection (CWE-89) and broken access control (CWE-862) (MITRE, n.d.), and it flags personal-information exposure against the Protection of Personal Information Act (Republic of South Africa, 2013). The design follows the detect, repair and validate pattern of recent work (Fakih et al., 2025; Gajjar et al., 2025).

The target codebase is the author's repository (github.com/Ngandana/African-Arts-Craft, branch master: Spring Boot 3.2.5, 127 Java files), called the 'real repository' below. Because it contains few defects of each kind and no independently labelled ground truth, the evaluation also uses a fixture of injected defects with known locations (Section 4.1, Appendix A). All results come from commands that can be re-run (Appendix A) and from saved result files.

This is a single-author project evaluated on one real repository and one injected fixture. The results show that the design works and quantify specific behaviours; they do not establish general performance across the industry.

2. Implementation Summary [35 Marks]

This section carries 35 marks: Code Structure, Adherence to Design & Best Practices (18) and Functionality Completeness Against Requirements (17).

2.1 Adherence to Approved Design

Explain how the implemented architecture and module structure follows the approved Design Phase HLD/DDD. Explicitly document and justify any deviations from the approved design.

The implementation follows the four-tier, five-module architecture of the Design Phase (Section 3.1 and Figure 1 of that document). Table 1 maps each designed component to its code module and Table 2 lists every deviation with its justification.

Design element (Design Phase)	Implemented in	Notes
Module 1: Scan Orchestrator	backend/app/orchestrator.py (run_scan, validate_scope)	Scope validation restricts scans to allowed roots that are git repositories
Module 2: Static Analysis Wrapper	backend/app/scanner.py	Built-in rules for the four CWE classes plus config-file secrets; optional Semgrep adapter and rules/legacyguard.yml (untested)
Module 3: LLM Triage Agent	backend/app/triage.py	Deterministic templates first; LLM (Claude API or any OpenAI-compatible endpoint, e.g. local Ollama) explains and, for unfamiliar cases, patches
Module 4: Remediation Staging and Verification	backend/app/staging.py, backend/app/build.py	Git worktree on an isolated branch; re-scan; build and test gate (added, Section 2.2)
Module 5: Human Review and Approval	orchestrator.decide; frontend/src/App.jsx	Only code path that merges; requires a reviewer and an approving Review record
Audit Trail and Logging	backend/app/audit.py; models.AuditLogEntry	SHA-256 hash chain plus ORM guards against update/delete
Data store, 8-entity 3NF schema	backend/app/models.py, db.py	All eight entities as designed; added Finding.rule_id, line_hash, status and Fix.popia_flag, branch
Interface specification (Design Phase 3.2.3)	backend/app/main.py	All five endpoints implemented; two added: /findings/groups, /audit/verify
Reviewer dashboard	frontend/src (React, Vite, Tailwind)	Findings list, detail, decision buttons, audit trail; root-cause panel added

Table 1. Mapping of designed components to implemented modules.

Deviation from design	Justification
SQLite is the default database; PostgreSQL is selected with DATABASE_URL	Zero-setup local runs and fast tests. The schema uses only portable SQLAlchemy types. PostgreSQL was not exercised.
Built-in line-based rules are the primary detector; Semgrep is optional	Semgrep is not available natively on the author's Windows machine. Detection quality is measured for the built-in engine only.
Template-first fixing; the LLM is secondary	Testing evidence (DEF-04, DEF-05): a 7B local model produced a wrong fix that passed re-scan, and failed on fixes a template handles.
Findings that need a design decision are never auto-patched	Plaintext password lookups and controller-wide missing authorisation need architectural change; a one-line patch would be misleading.
Test directories are excluded from scanning	Test fixtures were reported as production credentials (DEF-01).

Table 2. Deviations from the approved design, with justification.

2.2 Requirements Implementation Status

List every functional requirement from the Analysis/Design phases and its current implementation status. Use the same requirement IDs used in your Design Phase traceability matrix.

Requirement ID & Description	Status	Notes / Evidence Location

Requirement ID & Description	Status	Notes / Evidence Location
REQ-001: Detect four vulnerability classes	Implemented	backend/app/scanner.py. Table 8: precision and recall per CWE (line-based, not data-flow). Real repository: 20 findings in 3 root causes.
REQ-002: Plain-language explanation and POPIA flag	Implemented	backend/app/triage.py. The POPIA flag is a keyword heuristic (none / possible / likely), not a mapping to POPIA sections.
REQ-003: Specific, testable fix per finding	Partially Implemented	Verified fixes for hard-coded credentials, config secrets and SQL injection. Password-storage and authorisation findings are explained and escalated (Table 11).
REQ-004: Verify fix before the reviewer sees it	Implemented	staging.py and build.py: re-scan plus build and test gate (TC-009 to TC-011, EXP-B).
REQ-005: Explicit human approval before merge	Implemented	orchestrator.decide; TC-004, TC-005: target branch unchanged until an approving reviewer decision.
REQ-006: Approve, reject, request changes	Implemented	POST /findings/{id}/decision in main.py; TC-005.
REQ-007: Permanent, tamper-evident audit trail	Implemented	audit.py; TC-006. Immutability is enforced in the ORM; a raw SQL edit is detected, not prevented (OPEN-04).
REQ-008: Bounded retries, then escalation	Implemented	orchestrator.triage_and_fix; TC-010: exactly three attempts, then an 'escalated' audit entry.
REQ-009: Explicit actor permissions	Partially Implemented	Only decisions are role-restricted (reviewer). Scan and audit viewing are open to any user (OPEN-03).
NFR-001: No writes outside target or isolated branch	Implemented	TC-003, TC-007: base branch hash unchanged; path traversal and out-of-scope paths rejected.
NFR-002: Surface POPIA relevance	Implemented	popia_flag on every finding; real repository: 2 likely, 7 possible, 11 none.
NFR-003: Demonstrable on a real codebase	Implemented	EXP-D: scan of the author's repository, 3 s template-only, 147 s with local-LLM explanations.
NFR-004: Attributable scans and decisions	Partially Implemented	User foreign keys implemented in the schema. Identity is a seeded X-User-Id header, not real authentication (OPEN-03).

Table 3. Requirement implementation status. 'Implemented' means implemented and covered by an executed test or experiment.

Scope changes made during implementation (documented and justified):

- **Build and test gate added to verification.** The Design Phase verified a fix by re-running the same static analyser that produced the finding. That is circular: a patch that deletes the vulnerable line, or breaks the build, clears the scanner. It was observed directly (DEF-04). SecureFixAgent, cited in the Design Phase, closes its loop the same way (Gajjar et al., 2025), so adding compilation and regression tests is a deliberate extension of that pattern.
- **Findings grouped by root cause.** The first real scan produced 15 near-identical findings (one per controller), realising the 'reviewer fatigue' risk in the Design Phase risk table. The dashboard and GET /findings/groups now present one entry per root cause.
- **Local-first LLM option.** The problem statement cites unreliable electricity, limited connectivity and cost. A pipeline needing a metered cloud API for every scan contradicts that, so the agent accepts any OpenAI-compatible endpoint and was evaluated with a local 7B model (Section 4.1, Experiment C).
- **Dependency-aware fixing.** Templates no longer propose code that needs a library the project does not declare (DEF-09).

2.3 Code Structure & Best Practices

Describe how the codebase is organised (modules/packages/classes), naming and style conventions followed, and any best practices applied (e.g. version control workflow, error handling, dependency management).

The repository has three top-level parts: backend (FastAPI application in backend/app, tests in backend/tests, evaluation harness in backend/eval), frontend (React, Vite and Tailwind dashboard) and docs. Each backend module has one responsibility and none imports a later pipeline stage: scanner, triage, staging, build, orchestrator, audit and the API layer. Names follow Python conventions (snake_case functions, PascalCase ORM classes) and the SQLAlchemy 2.0 typed models mirror the 8-entity design.

Practices applied: configuration is environment-driven (backend/app/config.py) and secrets are never stored in source; dependencies are pinned in backend/requirements.txt and frontend/package.json; errors are explicit (Forbidden, Conflict and StagingError map to HTTP 403 and 409) and an LLM failure degrades to the deterministic result rather than blocking the pipeline; the staging module contains no merge function, so the human-in-the-loop rule is enforced structurally rather than by convention; the evaluation harness is kept separate from production code. Source is kept under version control at the repository named on the cover page.

3. Testing Strategy & Execution [30 Marks]

This section carries 30 marks: Test Plan Design & Coverage (15) and Test Execution, Results & Defect Management (15).

3.1 Test Plan

Define your test cases, each traced to a specific functional or non-functional requirement. Include unit, integration, and (where applicable) system/user acceptance test levels.

Three levels are used. (1) Unit and integration tests exercise the pipeline through the real HTTP API against a temporary git repository (pytest and FastAPI TestClient with real git operations; only Maven is stubbed in the gate tests for speed). (2) System-level experiments run the complete pipeline with real Maven builds on the real repository and on an injected-defect fixture. (3) User acceptance testing was not conducted with external users; the reviewer workflow was exercised through the API (Section 4.2).

Test Case ID	Requirement(s) Traced	Test Level	Test Description	Expected Result
TC-001	REQ-001	Integration	Scan the sample project containing all four weakness classes	5 findings covering CWE-798, 916, 89 and 862
TC-002	REQ-001	Integration	Group findings by root cause	One group per rule; locations sum to the findings
TC-003	REQ-004, REQ-005, NFR-001	Integration	Stage fixes and check the target branch	Every fix verified on its own branch; base branch hash unchanged
TC-004	NFR-001	Integration	Search API output and audit log for the credential value	Secret value appears nowhere
TC-005	REQ-005, REQ-006, REQ-009	Integration	Developer approves; reviewer approves, rejects, requests changes	Developer gets 403; approve merges; reject leaves target untouched; request-changes re-triages
TC-006	REQ-007, NFR-004	Integration	Verify audit order and hash chain; attempt ORM update; edit a row with raw SQL	Chain valid; ORM update blocked; tampering detected at the edited row
TC-007	NFR-001	Unit	Scan an out-of-scope path and a path-traversal path	Both rejected (HTTP 403)
TC-008	REQ-005	Integration	Approve a finding with no verified fix	HTTP 409, no merge
TC-009	REQ-004	Integration (Maven stubbed)	Patch that breaks compilation	Fix not verified; approval refused
TC-010	REQ-008	Integration (Maven stubbed)	Patch rejected on every attempt	Exactly three attempts, then an 'escalated' audit entry

Test Case ID	Requirement(s) Traced	Test Level	Test Description	Expected Result
TC-011	REQ-004	Integration (Maven stubbed)	Patch that breaks existing tests; green build; broken baseline	Broken tests rejected; green build verified; broken baseline does not block fixes (with warning)
EXP-A	REQ-001	System experiment	Detection precision and recall against injected ground truth (13 defects, 6 decoys)	High precision and recall per CWE class
EXP-B	REQ-003, REQ-004	System experiment	Full pipeline with the real Maven build gate on the injected fixture	Fixes that do not build are rejected, not presented as verified
EXP-C	REQ-003	System experiment	Template versus local LLM as patch author, same findings, same gate	LLM at least as reliable as templates (hypothesis from the design)
EXP-D	NFR-003, REQ-002, NFR-002	System experiment	Scan of the real repository, template-only and with local-LLM explanations	Findings, explanations and POPIA flags produced end to end

Table 4. Test plan with traceability to requirements.

Coverage. The automated suite measures line coverage of the application package at 83% (656 statements, 112 missed). The lowest coverage is in triage.py (62%), because the Claude and OpenAI-compatible call paths are exercised only by the experiments, and in scanner.py (75%), where the Semgrep adapter is unexecuted. Those are the paths with least automated protection.

3.2 Test Execution Results

Report the actual outcome of each test case executed. Attach logs, screenshots, or output data as evidence in the Appendices.

The suite was executed on 7 November 2026 on Windows 11 with Python 3.12.10, JDK 17.0.12 and Maven 3.10.0: 12 passed, 0 failed, in 22 seconds (log in Appendix A). The gate tests (TC-009 to TC-011) stub Maven; the real Maven gate is exercised in EXP-B.

Test Case ID	Actual Result	Pass / Fail	Evidence Reference (Appendix)
TC-001	5 findings, one per expected class	Pass	Appendix A, test_detects_all_four_classes
TC-002	Groups returned; location counts equal findings	Pass	Appendix A, test_findings_grouped_by_root_cause
TC-003	All 5 fixes verified on branches; main unchanged	Pass	Appendix A, test_fixes_verified_on_branch_and_target_untouched
TC-004	Secret not found in API responses or audit log	Pass	Appendix A, test_secret_never_stored_or_shown
TC-005	403 for developer; approve merged; reject and request-changes behaved as specified	Pass	Appendix A, test_approve_merges_and_audit_trail; test_reject_does_not_merge_and_request_changes_retriages
TC-006	Chain valid; ORM update raised; raw-SQL edit detected	Pass	Appendix A, test_audit_log_is_immutable_and_tamper_evident
TC-007	Both paths rejected	Pass	Appendix A, test_scope_and_path_traversal_rejected

Test Case ID	Actual Result	Pass / Fail	Evidence Reference (Appendix)
TC-008	409 returned, no merge	Pass	Appendix A, test_patch_that_breaks_compile_is_rejected; test_reject_does_not_merge...
TC-009	Fix marked build_failed; approval refused	Pass	Appendix A, test_patch_that_breaks_compile_is_rejected
TC-010	Three attempts then 'escalated' entry	Pass	Appendix A, test_patch_that_breaks_compile_is_rejected
TC-011	Broken tests rejected; green build verified; broken baseline warned	Pass	Appendix A, test_patch_that_breaks_tests_is_rejected; test_green_build_is_verified; test_broken_baseline_does_not_block_fixes
EXP-A	Initial rules: precision 0.86, recall 0.92 (12 TP, 2 FP, 1 FN). After DEF-10 and DEF-11: 13 TP, 0 FP, 0 FN	Fail, then Pass after fixes	Table 8, Figure 1; backend/eval/results_before.json, results_after.json
EXP-B	Scanner-only accepted 12 of 12; real gate verified 3 and rejected 9. After DEF-07 to DEF-09: 6 verified, 7 escalated	Fail, then Pass after fixes	Table 9, Figure 2; results_after.json
EXP-C	Template: 6 of 12 verified. Local 7B LLM: 2 of 12 verified, 6 build failures, 3 no patch, 1 still flagged	Fail (hypothesis not supported; design changed to template-first)	Table 10, Figure 3; results_after.json
EXP-D	20 findings in 3 root causes; 3 verified, 17 escalated; 3 s template-only, 147 s with LLM	Pass	Table 11; real_template.json, real_llm.json

Table 5. Execution results for every test case.

Module (backend/app)	Statements	Missed	Coverage
audit.py	27	0	100%
config.py	13	0	100%
models.py	86	1	99%
staging.py	71	3	96%
main.py	88	7	92%
orchestrator.py	106	15	86%
db.py	23	4	83%
scanner.py	96	24	75%
triage.py	95	36	62%
build.py	51	22	57%
Total	**656**	**112**	**83%**

Table 6. Line coverage reported by pytest-cov (Appendix A).

3.3 Defect Log & Resolution

For every failed test or defect discovered, record it here with its priority and current resolution status.

Defects were logged as found. Several were found by the experiments rather than by the unit tests, which is itself a finding about test design: the unit tests used a simple sample project and could not expose problems that depend on a real project's dependencies.

Defect ID	Description	Priority (H/M/L)	Status / Resolution
DEF-001	Test-class fixtures reported as production credentials (6 false positives on the real repository; found by EXP-D)	M	Fixed: test directories excluded
DEF-002	Secrets in application.properties not detected (false negative; found by manual inspection of EXP-D)	H	Fixed: rule lg-config-secret (3 findings)
DEF-003	Plaintext password comparison and missing controller authorisation not covered by rules	H	Fixed: rules lg-plaintext-password-lookup and lg-controller-no-authz
DEF-004	LLM fix for missing authorisation replaced the @RequestMapping line, removing the route, yet passed re-scan verification (local-LLM trial)	H (critical)	Fixed: build gate; design-level rules never auto-patched. Residual risk in Section 4.2
DEF-005	Local 7B model failed to patch config secrets that a template patches	M	Fixed: template first, LLM explains
DEF-006	7B model explained CORS instead of the actual issue	L	Fixed: prompt constrained to the rule
DEF-007	SQL template used PreparedStatement without an import (3 of 3 SQL fixes rejected by EXP-B gate)	H	Fixed: fully qualified type
DEF-008	SQL template assigned an int update result to a ResultSet	H	Fixed: original declared type preserved
DEF-009	Password-hashing and authorisation templates assumed Spring Security, absent from the project (6 rejections)	H	Fixed: dependency-aware templates; escalate with explanation
DEF-010	Decoy strings ('Password', 'Bearer') flagged as credentials (2 false positives; EXP-A)	M	Fixed with a label heuristic; trade-off in Section 4.2
DEF-011	Class-level authorisation check was file-wide, missing a class with one protected method (1 false negative; EXP-A)	M	Fixed: class block and unprotected-method check
DEF-012	Build gate silently skipped when the baseline does not compile (EXP-B first run)	M	Mitigated: explicit warning in the audit entry. Open: the result is still shown as verified
DEF-013	Evaluation harness inserted its dependency into the dependencyManagement block	L	Fixed
DEF-014	Test suite failed (6 of 12) when run in a shell where run.ps1 had left LEGACYGUARD_BASE_BRANCH=master set	M	Fixed: tests/conftest.py sets a fixed environment; run.ps1 clears its settings on exit
OPEN-01	The real repository's own test suite does not compile at baseline (stale AddressServiceTest, 2 errors), so regression tests cannot gate fixes there. Reported, not repaired: it is the target, not the tool	M	Open (deferred)
OPEN-02	Verification proves the project compiles and tests pass, not that a fix is secure; a weak but compilable password hash would pass	H	Open (deferred; needs security-specific checks)
OPEN-03	Authentication is a placeholder header; roles are enforced on decisions only	H	Open (deferred; unacceptable beyond a demonstration)
OPEN-04	Audit immutability enforced in the ORM; PostgreSQL triggers or revoked UPDATE/DELETE not implemented	M	Open (deferred)
OPEN-05	Semgrep adapter and PostgreSQL are untested	L	Open (deferred)
OPEN-06	application.properties in the real repository contains populated secret values committed to git history. Rotation is recommended; history was not altered	H	Open (repository owner action)

Table 7. Defect log (DEF) and open issues (OPEN) with priority and resolution status.

4. Evaluation & Results Analysis [25 Marks]

This section carries 25 marks: Presentation & Analysis of Evaluation Results (13) and Critical Evaluation Against Project Objectives (12).

4.1 Evaluation Results

Present the evidence used to judge system quality: performance metrics, benchmark comparisons, user feedback, or other evidence appropriate to your project type. Use tables/charts and reference them from the Appendices where large.

Experiment A: detection accuracy.

The fixture adds two Java files to a clone of the real repository: 13 injected defects (marked GT in the source, with known lines) and 6 decoys (marked TN: safe code resembling a defect, such as a parameterised query, an environment-variable password and an authorised admin endpoint). Findings that existed before injection are excluded. A finding counts as a true positive only at the exact marked line and CWE.

CWE	Initial rules TP/FP/FN	Precision / recall	After DEF-010, DEF-011 TP/FP/FN	Precision / recall
CWE-798	3/2/0	0.60 / 1.00	3/0/0	1.00 / 1.00
CWE-89	3/0/0	1.00 / 1.00	3/0/0	1.00 / 1.00
CWE-916	3/0/0	1.00 / 1.00	3/0/0	1.00 / 1.00
CWE-862	3/0/1	1.00 / 0.75	4/0/0	1.00 / 1.00
All	**12/2/1**	**0.86 / 0.92**	**13/0/0**	**1.00 / 1.00**

Table 8. Detection results on the injected fixture before and after the two heuristic fixes.

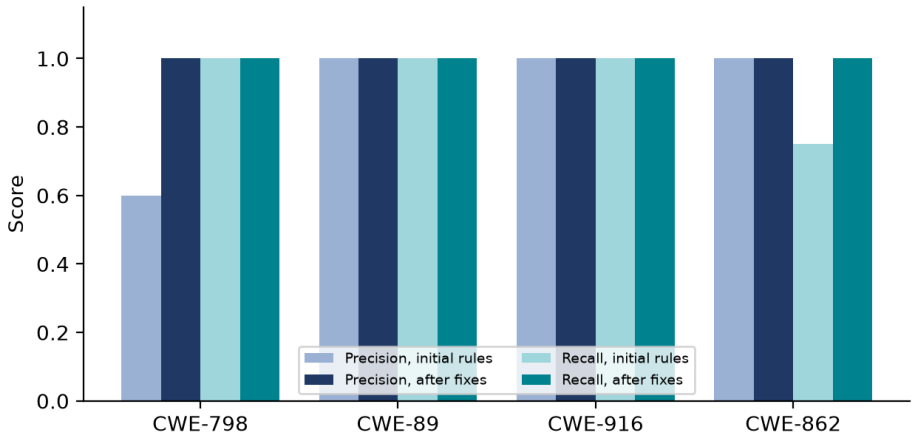


Figure 1. Precision and recall by CWE class, initial rules versus after fixes (injected fixture).

Interpretation. With the initial rules, two decoy strings were flagged and one class-level authorisation gap was missed. After fixing both, all 13 defects were found and no decoy was flagged. This result is optimistic and must not be read as the tool's true accuracy: the two fixes were designed after seeing the failures on this same fixture, the fixture is small, and its ground truth was written by the same author as the rules. A held-out fixture written by a second person is needed for a general claim. The label heuristic also has a cost: a real hard-coded password that is purely alphabetic and under 16 characters (for example 'letmein') is now ignored.

Experiment B: effect of the build and test gate.

Verification method	Verified	Rejected by gate	Escalated (no auto-fix)
Scanner re-scan only (original design)	12 of 12	0	0
With build gate, initial templates	3 of 12	9	0

Verification method	Verified	Rejected by gate	Escalated (no auto-fix)
With build gate, after DEF-007 to DEF-009 fixes (13 findings incl. class-level)	6 of 13	0	7

Table 9. Fixes accepted by each verification method on the injected fixture (real Maven compile on JDK 17).

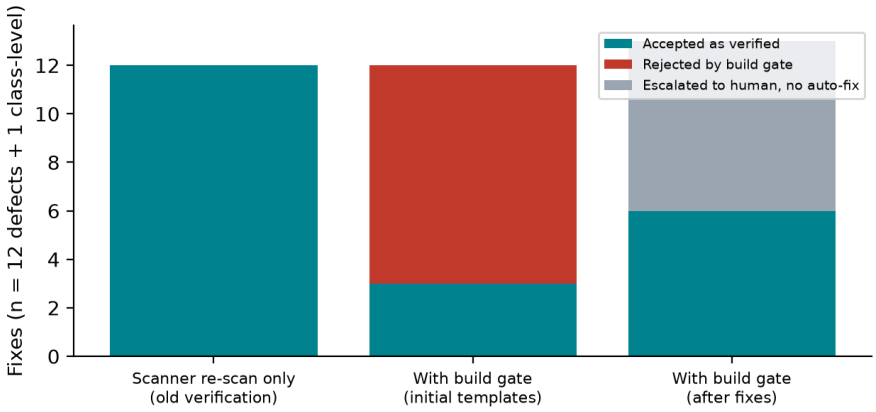


Figure 2. Outcome of verification by method. The scanner-only method accepted every fix.

Interpretation. The original method accepted all 12 proposed fixes. The compile gate showed that 9 did not build, which exposed three defects in the templates themselves (DEF-007 to DEF-009). After those fixes, 6 fixes are verified (all hard-coded credential and SQL injection fixes) and the 7 cases needing a dependency or a design decision are escalated to the reviewer with an explanation rather than presented as fixes. The first run also showed the gate being inert when the baseline did not compile (DEF-012), which is why that condition now produces a warning.

Experiment C: local LLM versus deterministic templates.

The same 12 auto-patchable findings were patched twice: by the deterministic template and by a local model (qwen2.5-coder 7B on Ollama, CPU only, no network). Both patches passed through the same re-scan and build gate. The template run had no dependency check, to isolate what the model does.

Patch author	Verified	Rejected by build gate	Other	Mean generation time
Deterministic template	6 of 12	6	0	under 0.1 s
Local LLM, 7B, CPU	2 of 12	6	3 no patch, 1 still flagged	8.7 s (max 13.4 s)

Table 10. Template versus local LLM as patch author (same findings, same verification).

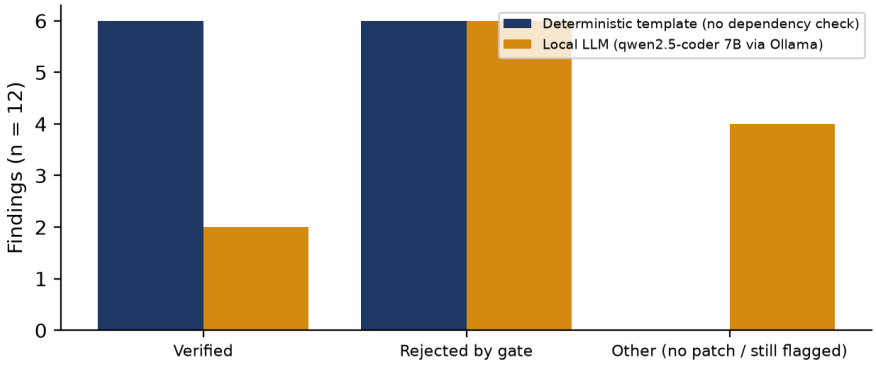


Figure 3. Verified versus rejected fixes by patch author (n = 12).

Interpretation. On this small sample the 7B model was clearly weaker than a template for patterns that have a known safe form, and slower. It did produce verified fixes for two password-hashing findings that the dependency-blind template could not, so it is not useless; but without a human reviewer and a gate its output

would not be trustworthy (DEF-004). These results support using the model for explanation and for cases without a template, behind the gate. They do not show that a larger local model, or the Claude API, would behave the same way; neither was benchmarked. About 8.7 s per call on an ordinary laptop CPU is acceptable for an on-demand scan and removes both the per-scan cost and the need for connectivity.

Experiment D: the real repository.

Root cause (rule)	Findings	Outcome
Secrets committed in configuration (lg-config-secret)	3	3 fixes verified (replaced by environment placeholders); build gate not applicable to .properties files
Passwords stored and compared in plaintext (lg-plaintext-password-lookup)	2	Explained; escalated (design change)
No authorisation on controllers (lg-controller-no-authz)	15	Explained; escalated (no security layer or dependency in the project)
Total	**20**	**3 verified; reviewer sees 3 root causes instead of 20 items; 3 s template-only, 147 s with local-LLM explanations**

Table 11. Scan of the author's repository with the final implementation.

Interpretation. Two things matter more than the count. First, 17 of 20 findings could not be auto-fixed, because the dominant weakness in this codebase is an absent control (no authentication or authorisation layer, no password hashing) rather than a bad line of code. Pattern-based detection finds present patterns well and absences poorly: the tool reports the symptom (every controller is open) but cannot say what to build. Second, grouping reduced the reviewer's queue from 20 items to 3, which addresses the reviewer-fatigue risk identified in the Design Phase; whether this makes review faster was not measured. The POPIA heuristic flagged 9 of 20 findings (2 likely, 7 possible); with no labelled set, its accuracy is unknown.

4.2 Critical Evaluation Against Project Objectives

Revisit each objective stated in your Design/Analysis Phase documents. For each, state honestly whether it was met, partially met, or not met, and why.

Project Objective	Met / Partially Met / Not Met	Explanation
1. Agentic pipeline that detects and explains within four CWE classes on a real legacy codebase	Met (with limits)	End to end on the real repository (Table 11); detection precision and recall in Table 8. Limits: line-based rules, no data-flow analysis, optimistic fixture result, Semgrep untested, explanation quality not rated by developers.
2. Iterative remediation that proposes and verifies fixes without merging autonomously	Partially Met	Bounded three-attempt loop, isolated branches, no merge path without a reviewer (TC-003, TC-005, TC-010). Verified fixes exist for 3 of the 5 defect types; build gate added. Shortfall: verification confirms buildability, not security (OPEN-02), and retries of a deterministic template repeat the same answer.
3. Transparent, immutable audit trail addressing the trust barrier (TechCabal, 2026)	Partially Met	Hash-chained, ordered, per-finding trail with tamper detection (TC-006). Shortfall: tamper-evident rather than tamper-proof (OPEN-04); trust itself was not tested with South African developers, so the claim that it addresses the adoption barrier is untested.
4. Working prototype and walkthrough of the review-and-approval workflow	Met	React dashboard and API replace the static wireframes, and the full review workflow is covered by the API tests. Shortfall: no usability test was run, and authentication is a placeholder (OPEN-03).
5. Complete traceability matrix linking requirements to design elements	Met	Tables 3 and 4 extend the Design Phase matrix to implementation and to test cases. Three requirements are only partly met (REQ-003, REQ-009, NFR-004).

Table 12. Assessment of each project objective.

What worked.

- Separating detection, reasoning and action made faults attributable: every logged defect was traced to a specific stage.
- The build gate was the most valuable change: it converted 9 plausible but broken fixes into rejections and found three template defects the unit tests missed.
- Template-first fixing with the LLM behind a gate gave predictable results, and local execution is practical on an ordinary laptop.

What did not work, or is unproven.

- A 7B local model is not reliable enough to author fixes unsupervised; one of its fixes deleted a route and passed the scanner (DEF-004).
- Most real findings (17 of 20) were not auto-fixable, which limits how much effort the tool saves on a codebase like this one.
- No user study was done, so the claims about reviewer fatigue and trust remain design arguments supported only by the grouping and audit mechanisms existing.

Threats to validity.

- **Construct:** 'Verified' means re-scan clear plus compile (and tests where runnable). On the real repository the existing tests do not compile, so no regression testing occurred there.
- **Internal:** Heuristics were tuned on the same fixture they were then scored on (Table 8); ground truth and rules share one author.
- **External:** One real repository, one fixture, one local model and a single run each (the experiments are deterministic except for LLM output, run once at temperature 0).
- **Residual semantic risk:** A compilable but weaker fix (for example a different fast hash) would pass the gate.

5. Implementation & Testing Documentation [10 Marks]

This section carries 10 marks: Technical Documentation Quality (10).

5.1 Setup & Installation Instructions

Provide instructions sufficient for another developer to install dependencies, configure, and run the system from the submitted repository/archive.

- **Prerequisites:** Python 3.12, Node 24 and Git. For the build gate: JDK 17 and Maven 3.10 with JAVA_HOME and MAVEN_HOME set. Optional: Ollama (ollama pull qwen2.5-coder:7b).
- **Backend and tests:** cd backend; python -m venv .venv; .\.venv\Scripts\pip install -r requirements.txt; .\.venv\Scripts\python -m pytest -q (expects 12 passed).
- **Frontend:** cd frontend; npm install. On Windows, if scripts are blocked run Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned once.
- **Run:** From the repository root run .\run.ps1. It starts the API on port 8000 and the dashboard on port 5173 (http://localhost:5173). Paste the path of the target repository and select Trigger scan. The default target branch is main; run.ps1 sets LEGACYGUARD_BASE_BRANCH=master for the author's repository.
- **LLM options:** Free local LLM: set LLM_BASE_URL=http://localhost:11434/v1 and LLM_MODEL=qwen2.5-coder:7b before run.ps1. Paid alternative: ANTHROPIC_API_KEY. Without either, templates are used.
- **Database:** SQLite by default; DATABASE_URL selects PostgreSQL (install psycopg).
- **Reproducing the evaluation:** The commands in Appendix A reproduce every experiment; raw outputs are in backend/eval/results_*.json.

5.2 Key Technical Decisions

Summarise any significant technical decisions made during implementation that were not already covered in the Design Phase (e.g. library substitutions, workarounds for constraints discovered during development).

- Deterministic templates run before LLM output, so a known safe pattern is never at the mercy of a small model.
- Secrets are redacted at the scanner, so the redacted form is all that is stored, displayed, logged or sent to a model; the original line is re-read locally from git only when a patch is applied.
- Fixes are located by the SHA-256 of the original line rather than by line number, so earlier merges that shift lines do not misplace a later patch.
- The staging module has no merge capability; merging exists only in orchestrator.decide behind a reviewer check.
- Findings needing design decisions are explained but never auto-patched.
- The build gate is differential: it compares against the unmodified base commit (cached per commit), so a repository that is already broken is reported honestly instead of blocking every fix.
- **Extending the system:** To add a vulnerability class: add a detection rule in scanner.py and a CWECategory row in db.py; optionally add a template in triage._offline_fix and, if the fix needs a library, a flag in triage.capabilities. No change to the orchestrator, staging or API is required.

6. Self-Reflection

This reflects Section E of the rubric (holistic, no independent marks, but informs overall mark quality): Code Quality, Security & Maintainability; Academic Writing, Referencing & Presentation.

Security measures implemented.

- Credential redaction at the source (TC-004): no secret is stored in the database, audit log, API response or LLM prompt.
- Scope control: scans are limited to allowed roots that are git repositories, and file paths are resolved and must stay inside the checkout (TC-007).
- The target branch is changed only by an approved reviewer decision; fixes live on isolated branches.
- Reviewer-only decision endpoint; every scan and review is attributed to a user in the schema.
- Hash-chained audit log with an integrity endpoint (GET /audit/verify).

Known weaknesses.

- Authentication is a trusted request header: anyone who can reach the API can claim to be the reviewer. This is unacceptable outside a demonstration.
- Audit immutability is not enforced by the database; a privileged user can alter rows, which the chain would detect but not prevent.
- Cross-origin access is restricted to the local dashboard origin, but there is no rate limiting or request size limiting.
- Semgrep, PostgreSQL and the Claude API path were not exercised.
- Maintainability: the scanner's regular expressions are dense and the Java templates are string-based; a parser-based approach would be more robust for a second language or framework.

Reflection. The most significant result is negative evidence for the original design: verifying a fix by re-running the scanner is circular, and adding a build gate changed the acceptance of fixes from 12 of 12 to 6 of 13, with the remainder escalated honestly. The tool is useful as a triage and review aid; it is not a substitute for a security engineer, and on the author's own repository most findings reflect missing controls that no automated patch should supply. Recommended next work: a held-out evaluation fixture written by a second person, real authentication, database-level audit protection, and a small usability test with developers.

Sources are cited in the Harvard style and listed in Section 7; the citations support the design pattern and the problem context, not the measured results, which come from the experiments reported here.

7. References

List all sources cited in this document using the institution's prescribed referencing style (typically Harvard or APA 7th edition).

Fakih, M., Dharmaji, R., Bouzidi, H., Quiros Araya, G., Ogundare, O. and Al Faruque, M.A. (2025) LLM4CVE: enabling iterative automated vulnerability repair with large language models. arXiv:2501.03446. Available at: https://arxiv.org/pdf/2501.03446 (Accessed: 2 August 2026).

Gajjar, J., Subramaniakuppusamy, K., Puthal, R. and Ranaware, K. (2025) SecureFixAgent: a hybrid LLM agent for automated Python static vulnerability repair. arXiv:2509.16275. Available at: https://arxiv.org/pdf/2509.16275 (Accessed: 1 August 2026).

MITRE (n.d.) Common Weakness Enumeration: CWE-89, CWE-798, CWE-862, CWE-916. Available at: https://cwe.mitre.org (Accessed: 7 November 2026).

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.

TechCabal (2026) South Africa's AI skills gap is widening faster than universities can adapt. Available at: https://techcabal.com (Accessed: 27 July 2026).

8. Appendices

Appendix A: Test Execution Evidence

Insert test logs, screenshots, or output data referenced in Section 3.2.

A.1 Automated test log (python -m pytest -v --cov=app, backend directory, 7 November 2026). Full file: docs/evidence/pytest_output.txt.

```
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\USER-
PC\OneDrive\projects\legacyguard\backend
rootdir: C:\Users\USER-PC\OneDrive\projects\legacyguard\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.16.0, cov-7.1.0
collecting ... collected 12 items
tests/test_gate.py::test_patch_that_breaks_compile_is_rejected PASSED    [  8%]
tests/test_gate.py::test_patch_that_breaks_tests_is_rejected PASSED      [ 16%]
tests/test_gate.py::test_green_build_is_verified PASSED                  [ 25%]
tests/test_gate.py::test_broken_baseline_does_not_block_fixes PASSED     [ 33%]
tests/test_pipeline.py::test_detects_all_four_classes PASSED             [ 41%]
tests/test_pipeline.py::test_fixes_verified_on_branch_and_target_untouched PASSED [ 50%]
tests/test_pipeline.py::test_secret_never_stored_or_shown PASSED         [ 58%]
tests/test_pipeline.py::test_approve_merges_and_audit_trail PASSED       [ 66%]
tests/test_pipeline.py::test_reject_does_not_merge_and_request_changes_retriages PASSED [ 75%]
tests/test_pipeline.py::test_audit_log_is_immutable_and_tamper_evident PASSED [ 83%]
tests/test_pipeline.py::test_scope_and_path_traversal_rejected PASSED    [ 91%]
tests/test_pipeline.py::test_findings_grouped_by_root_cause PASSED       [100%]
============================== warnings summary ===============================
app\main.py:17
.venv\Lib\site-packages\fastapi\applications.py:4723
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================ tests coverage ================================
_____________ coverage: platform win32, python 3.12.10-final-0 _____________
Name                   Stmts   Miss  Cover
------------------------------------------
app\audit.py              27      0   100%
app\build.py              51     22    57%
app\config.py             13      0   100%
app\db.py                 23      4    83%
app\main.py               88      7    92%
app\models.py             86      1    99%
app\orchestrator.py      106     15    86%
app\scanner.py            96     24    75%
app\staging.py            71      3    96%
app\triage.py             95     36    62%
------------------------------------------
TOTAL                    656    112    83%
```

A.2 Experiment commands and result files. From the backend directory: python -m eval.run_eval after --llm (writes eval/results_after.json; needs Ollama running for the LLM part); python -m eval.run_eval before (initial rules; eval/results_before.json); python -m eval.real_repo [--llm] (real repository; real_template.json,

real_llm.json). The fixture files are backend/eval/fixtures/InjectedDefects.java and InjectedAdminController.java; ground truth is carried in the source as '// GT:CWE-n' and decoys as '// TN'.

A.3 Charts. Figures 1 to 3 in Section 4.1 (source images: docs/evidence/fig_detection.png, fig_gate.png, fig_llm.png).

Appendix B: Representative Code Snippets

Include representative code snippets illustrating key functionality. All code must be formatted in a monospaced font with line numbers. Do not paste the entire codebase; reference the repository for full source.

B.1 Build and test gate (differential against the base commit)

Source: backend/app/build.py, lines 62 to 74

```
 62  def gate(root: Path, base: dict) -> dict:
 63      """Run on a patched checkout. Returns {ok, detail}."""
 64      if not base["compiles"]:
 65          return {"ok": True, "detail": "WARNING: baseline does not compile, so the build gate could
not verify this fix"}
 66      ok, tail = run_mvn(root, "compile")
 67      if not ok:
 68          return {"ok": False, "detail": "patched code no longer compiles: " + tail.replace("\n", "
")[:300]}
 69      if base["tests_runnable"]:
 70          ok, tail = run_mvn(root, "test")
 71          if not ok:
 72              return {"ok": False, "detail": "existing tests fail after the patch: " +
tail.replace("\n", " ")[:300]}
 73          return {"ok": True, "detail": "compiles and existing tests pass"}
 74      return {"ok": True, "detail": "compiles (baseline tests not runnable, so regression tests
skipped)"}
```

B.2 Human approval is the only merge path

Source: backend/app/orchestrator.py, lines 96 to 126

```
 96  def decide(db: Session, finding_id: int, reviewer: User, decision: str, feedback: str = "") ->
Finding:
 97      if reviewer.role != "reviewer":
 98          raise Forbidden("only a reviewer may decide")
 99      finding = db.get(Finding, finding_id)
100      if finding is None:
101          raise LookupError("finding not found")
102      if finding.status != "awaiting_review" or not finding.fixes:
103          raise Conflict(f"finding is {finding.status}, not awaiting review")
104      fix = finding.fixes[-1]
105      review = Review(fix_id=fix.fix_id, user_id=reviewer.user_id, decision=decision,
feedback=feedback)
106      db.add(review)
107      db.flush()
108      audit.log(db, "review", review.review_id, "reviewed", f"{decision} by {reviewer.name}",
reviewer.name,
109                finding.finding_id)
110      root = Path(db.get(Repository, finding.scan.repo_id).url)
111      if decision == "approve":
112          if fix.verification_result != "resolved" or not fix.branch:
113              db.rollback()
114              raise Conflict("cannot approve: no verified fix branch exists; reject or request
changes")
115          if staging.git(root, "rev-parse", "--abbrev-ref", "HEAD") != config.BASE_BRANCH:
116              db.rollback()
117              raise Conflict(f"target repository is not on {config.BASE_BRANCH}")
118          try:
119              staging.git(root, "-c", "user.name=LegacyGuard", "-c", "user.email=legacyguard@local",
"merge", "--no-ff",
120                          "-q", "-m", f"LegacyGuard: merge {fix.branch} (approved by {reviewer.name})",
fix.branch)
121          except staging.StagingError as e:
122              subprocess.run(["git", "-C", str(root), "merge", "--abort"], capture_output=True)
123              db.rollback()
124              raise Conflict(f"merge conflict; re-scan to regenerate this fix ({e})")
125          staging.git(root, "branch", "-D", fix.branch)
126          finding.status = "merged"
```

B.3 Hash-chained audit log and verification

Source: backend/app/audit.py, lines 16 to 44

```
 16  def _digest(prev, entity_type, entity_id, finding_id, action, detail, actor, ts) -> str:
 17      raw = "|".join(map(str, [prev, entity_type, entity_id, finding_id, action, detail, actor,
_canon(ts)]))
 18      return hashlib.sha256(raw.encode()).hexdigest()
 19
 20
 21  def log(db: Session, entity_type: str, entity_id: int, action: str, detail: str = "",
 22          actor: str = "system", finding_id: int | None = None) -> AuditLogEntry:
 23      last = db.execute(select(AuditLogEntry).order_by(AuditLogEntry.log_id.desc()).limit(1)).scalar()
 24      prev = last.entry_hash if last else GENESIS
 25      ts = utcnow().replace(microsecond=0)
 26      entry = AuditLogEntry(entity_type=entity_type, entity_id=entity_id, finding_id=finding_id,
action=action,
 27                            detail=detail, actor=actor, timestamp=ts, prev_hash=prev,
 28                            entry_hash=_digest(prev, entity_type, entity_id, finding_id, action,
detail, actor, ts))
 29      db.add(entry)
 30      db.flush()
 31      return entry
 32
 33
 34  def verify_chain(db: Session) -> dict:
 35      prev = GENESIS
 36      rows = db.execute(select(AuditLogEntry).order_by(AuditLogEntry.log_id)).scalars().all()
 37      for r in rows:
 38          expected = _digest(prev, r.entity_type, r.entity_id, r.finding_id, r.action, r.detail,
r.actor, r.timestamp)
 39          if r.prev_hash != prev or r.entry_hash != expected:
 40              return {"valid": False, "broken_at": r.log_id, "entries": len(rows)}
 41          prev = r.entry_hash
 42      return {"valid": True, "entries": len(rows)}
```

9. Self-Assessment Checklist: Rubric Alignment

Complete this checklist before submitting. Each criterion maps directly to the Implementation & Testing/Evaluation Rubric.

Section Reference	Assessment Criterion	Max Marks	Self-Score
Section 2	Implementation Summary	35	29 / 35
Section 3	Testing Strategy & Execution	30	25 / 30
Section 4	Evaluation & Results Analysis	25	20 / 25
Section 5	Implementation & Testing Documentation	10	8 / 10
	TOTAL	**100**	**82 / 100**

Criterion	Completed? (Yes / Partial / No)
Every requirement from the Design Phase traceability matrix has a recorded implementation status	Yes
Test cases are explicitly traced to specific requirements, across appropriate test levels	Yes (no external user acceptance testing)
Test execution results are reported honestly, including any failures, with evidence attached	Yes
A defect log is included with priority and resolution status for every issue found	Yes
Evaluation results are presented with appropriate evidence (metrics/charts/feedback)	Yes
Every project objective is critically assessed as met, partially met, or not met, with explanation	Yes

Criterion	Completed? (Yes / Partial / No)
Setup/installation instructions are complete enough for another developer to run the system	Yes
Document has been proofread and uses formal academic language throughout	Partial (self-proofread only)

End of Implementation & Testing/Evaluation Submission